Instalación Dependencias

In [ ]:

```
!pip install 'numpy<2.0'
```

```
Requirement already satisfied: numpy<2.0 in /usr/local/lib/python3.11/dist-packages (1.26.4)
```

In [ ]:

```
!pip install ydata-profiling
```

```
Requirement already satisfied: ydata-profiling in /usr/local/lib/python3.11/dist-packages (4
.16.1)
Requirement already satisfied: scipy<1.16,>=1.4.1 in /usr/local/lib/python3.11/dist-packages
(from ydata-profiling) (1.15.2)
Requirement already satisfied: pandas!=1.4.0,<3.0,>1.1 in /usr/local/lib/python3.11/dist-pac
kages (from ydata-profiling) (2.2.2)
Requirement already satisfied: matplotlib<=3.10,>=3.5 in /usr/local/lib/python3.11/dist-pack
ages (from ydata-profiling) (3.10.0)
Requirement already satisfied: pydantic>=2 in /usr/local/lib/python3.11/dist-packages (from
ydata-profiling) (2.11.3)
Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.11/dist-packages
(from ydata-profiling) (6.0.2)
Requirement already satisfied: jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.11/dist-package
s (from ydata-profiling) (3.1.6)
Requirement already satisfied: visions<0.8.2,>=0.7.5 in /usr/local/lib/python3.11/dist-packa
ges (from visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (0.8.1)
Requirement already satisfied: numpy<2.2,>=1.16.0 in /usr/local/lib/python3.11/dist-packages
(from ydata-profiling) (1.26.4)
Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.11/dist-packages (f
rom ydata-profiling) (0.1.12)
Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.11/dist-packages
(from ydata-profiling) (0.12.4)
Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.11/dist-package
s (from ydata-profiling) (2.32.3)
Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.11/dist-packages (f
rom ydata-profiling) (4.67.1)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in /usr/local/lib/python3.11/dist-packa
ges (from ydata-profiling) (0.13.2)
Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.11/dist-package
s (from ydata-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.11/dist-pack
ages (from ydata-profiling) (0.14.4)
Requirement already satisfied: typeguard<5,>=3 in /usr/local/lib/python3.11/dist-packages (f
rom ydata-profiling) (4.4.2)
Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.11/dist-packages (
from ydata-profiling) (4.3.1)
Requirement already satisfied: wordcloud>=1.9.3 in /usr/local/lib/python3.11/dist-packages (
from ydata-profiling) (1.9.4)
Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.11/dist-packages (from
ydata-profiling) (1.9.2)
Requirement already satisfied: numba<=0.61,>=0.56.0 in /usr/local/lib/python3.11/dist-packag
es (from ydata-profiling) (0.61.0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.11/dist-packages (from i
magehash==4.3.1->ydata-profiling) (1.8.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.11/dist-packages (from image
hash==4.3.1->ydata-profiling) (11.2.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (f
rom jinja2<3.2,>=2.11.1->ydata-profiling) (3.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (
from matplotlib<=3.10,>=3.5->ydata-profiling) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from
```

```
matplotlib<=3.10,>=3.5->ydata-profiling) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages
(from matplotlib<=3.10,>=3.5->ydata-profiling) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages
(from matplotlib<=3.10,>=3.5->ydata-profiling) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (f
rom matplotlib<=3.10,>=3.5->ydata-profiling) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (
from matplotlib<=3.10,>=3.5->ydata-profiling) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packag
es (from matplotlib<=3.10,>=3.5->ydata-profiling) (2.9.0.post0)
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in /usr/local/lib/python3.11/dist-
packages (from numba<=0.61,>=0.56.0->ydata-profiling) (0.44.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from
pandas!=1.4.0,<3.0,>1.1->ydata-profiling) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (fr
om pandas!=1.4.0,<3.0,>1.1->ydata-profiling) (2025.2)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.11/dist-packages (fr
om phik<0.13,>=0.11.1->ydata-profiling) (1.4.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-pack
ages (from pydantic>=2->ydata-profiling) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.1 in /usr/local/lib/python3.11/dist-packa
ges (from pydantic>=2->ydata-profiling) (2.33.1)
Requirement already satisfied: typing-extensions>=4.12.2 in /usr/local/lib/python3.11/dist-p
ackages (from pydantic>=2->ydata-profiling) (4.13.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-pa
ckages (from pydantic>=2->ydata-profiling) (0.4.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-pa
ckages (from requests<3,>=2.24.0->ydata-profiling) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from
requests<3,>=2.24.0->ydata-profiling) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages
(from requests<3,>=2.24.0->ydata-profiling) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages
(from requests<3,>=2.24.0->ydata-profiling) (2025.1.31)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-packages (from
statsmodels<1,>=0.13.2->ydata-profiling) (1.0.1)
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.11/dist-packages (fro
m visions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (25.3.0)
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.11/dist-packages (fro
m visions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (3.4.2)
Requirement already satisfied: puremagic in /usr/local/lib/python3.11/dist-packages (from vi
sions<0.8.2,>=0.7.5->visions[type_image_path]<0.8.2,>=0.7.5->ydata-profiling) (1.28)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from pyt
hon-dateutil>=2.7->matplotlib<=3.10,>=3.5->ydata-profiling) (1.17.0)
```

In [ ]:

```
!pip install scikit-surprise
```

```
Requirement already satisfied: scikit-surprise in /usr/local/lib/python3.11/dist-packages (1
.1.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (fro
m scikit-surprise) (1.4.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (fro
m scikit-surprise) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from
scikit-surprise) (1.15.2)
```

In [ ]:

```
!pip install rarfile
```

```
Requirement already satisfied: rarfile in /usr/local/lib/python3.11/dist-packages (4.2)
```

In [ ]:

```
!pip install folium
```

```
!pip install folium
```

```
Requirement already satisfied: folium in /usr/local/lib/python3.11/dist-packages (0.19.5)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.11/dist-packages (fro
m folium) (0.8.1)
Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.11/dist-packages (from
folium) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from folium
) (1.26.4)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from fol
ium) (2.32.3)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.11/dist-packages (from
folium) (2025.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (f
rom jinja2>=2.9->folium) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-pa
ckages (from requests->folium) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from
requests->folium) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages
(from requests->folium) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages
(from requests->folium) (2025.1.31)
```

Librerías

In [ ]:

```
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from surprise import SVD
from surprise import Reader
from surprise import Dataset
from surprise import accuracy
from surprise import KNNWithZScore
from surprise.model_selection import GridSearchCV

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import make_scorer, mean_squared_error
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics.pairwise import cosine_similarity

from concurrent.futures import ThreadPoolExecutor

import time
import os
import random
import warnings
import multiprocessing
import joblib
import gdown
from datetime import datetime

import zipfile # para descomprimir una carpeta *.zip
import tarfile # para descomprimir una carpeta *.tar *.gz
import rarfile # para descomprimir una carpeta *.rar

from ydata_profiling import ProfileReport # Perfilamiento de datos

%matplotlib inline
```

In [ ]:

```
# Para garantizar reproducibilidad en resultados, se define la semilla global
seed = 10
random.seed(seed)
np.random.seed(seed)
```

In [ ]:

```
# Configuración global
warnings.filterwarnings('ignore')
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
pd.set_option('display.float_format', '{:.3f}'.format) # Configuración global de presentaci
ón de .3 decimales en resul
```

In [ ]:

```
# Activar paralelismo
os.environ["OMP_NUM_THREADS"] = str(multiprocessing.cpu_count()) # Para NumPy y OpenMP
os.environ["MKL_NUM_THREADS"] = str(multiprocessing.cpu_count()) # Para librerías basadas en
MKL
os.environ["NUMEXPR_NUM_THREADS"] = str(multiprocessing.cpu_count()) # Para NumExpr
os.environ["OPENBLAS_NUM_THREADS"] = str(multiprocessing.cpu_count()) # Para OpenBLAS
os.environ["TF_NUM_INTRAOP_THREADS"] = str(multiprocessing.cpu_count()) # Para TensorFlow
os.environ["TF_NUM_INTEROP_THREADS"] = str(multiprocessing.cpu_count()) # Para TensorFlow
```

Funciones de apoyo

In [ ]:

```
# Función para descomprimir archivo
def descomprimir_archivo(archivo, directorio_destino):
    # Verificar la extensión del archivo y descomprimir según corresponda
    if archivo.endswith('.zip'):
        # Descomprimir un archivo ZIP
        with zipfile.ZipFile(archivo, 'r') as zip_ref:
            zip_ref.extractall(directorio_destino)
        print(f"Archivos extraídos de {archivo} en {directorio_destino}")

    elif archivo.endswith('.tar'):
        # Descomprimir un archivo TAR
        with tarfile.open(archivo, 'r') as tar_ref:
            tar_ref.extractall(directorio_destino)
        print(f"Archivos extraídos de {archivo} en {directorio_destino}")

    elif archivo.endswith('.tar.gz') or archivo.endswith('.tgz'):
        # Descomprimir un archivo TAR.GZ
        with tarfile.open(archivo, 'r:gz') as tar_ref:
            tar_ref.extractall(directorio_destino)
        print(f"Archivos extraídos de {archivo} en {directorio_destino}")

    elif archivo.endswith('.tar.bz2'):
        # Descomprimir un archivo TAR.BZ2
        with tarfile.open(archivo, 'r:bz2') as tar_ref:
            tar_ref.extractall(directorio_destino)
        print(f"Archivos extraídos de {archivo} en {directorio_destino}")

    elif archivo.endswith('.rar'):
        # Descomprimir un archivo RAR
        with rarfile.RarFile(archivo) as rar_ref:
            rar_ref.extractall(directorio_destino)
        print(f"Archivos extraídos de {archivo} en {directorio_destino}")

    else:
        print("Formato de archivo no soportado o desconocido.")
```

In [ ]:

```
# Función para capturar tiempos de ejecución
def timer(start_time=None):
  if not start_time:
    start_time = datetime.now()
    return start_time
  elif start_time:
    thour, temp_sec = divmod((datetime.now() - start_time).total_seconds(), 3600)
    tmin, tsec = divmod(temp_sec, 60)
    print('\n Time taken: %i hour(s) %i minute(s) and %s second(s).' % (thour, tmin, round(
tsec, 2)))
```

Diccionario de datos

business

Contains business data including location data, attributes, and categories.

#	name	type	description
1	business_id	string	22 character unique string business id
2	name	string	the business's name
3	address	string	the full address of the business
4	city	string	the city
5	state	string	2 character state code, if applicable
6	postal code	string	the postal code
7	latitude	float	latitude
8	longitude	float	longitude
9	stars	float	star rating, rounded to half-stars
10	review_count	integer	number of reviews
11	is_open	integer	0 or 1 for closed or open, respectively
13	attributes	object	business attributes to values. note: some attribute values might be objects
14	categories	array	an array of strings of business categories
15	hours	object	an object of key day to value hours, hours are using a 24hr clock

review

Contains full review text data including the user_id that wrote the review and the business_id the review is written for.

name	type	description
review_id	string	22 character unique review id
user_id	string	22 character unique user id, maps to the user in user.json
business_id	string	22 character business id, maps to business in business.json
stars	integer	star rating
date	string	date formatted YYYY-MM-DD
text	string	the review itself
useful	integer	number of useful votes received

name	type	description
funny	integer	number of funny votes received
cool	integer	number of cool votes received

user

User data including the user's friend mapping and all the metadata associated with the user.

name	type	description
user_id	string	22 character unique user id, maps to the user in user.json
name	string	the user's first name
review_count	integer	the number of reviews they've written
yelping_since	string	when the user joined Yelp, formatted like YYYY-MM-DD
friends	array	array of strings, an array of the user's friend as user_ids
useful	integer	number of useful votes sent by the user
funny	integer	number of funny votes sent by the user
cool	integer	number of cool votes sent by the user
fans	integer	number of fans the user has
elite	array	array of integers, the years the user was elite
average_stars	float	average rating of all reviews
compliment_hot	integer	number of hot compliments received by the user
compliment_more	integer	number of more compliments received by the user
compliment_profile	integer	number of profile compliments received by the user
compliment_cute	integer	number of cute compliments received by the user
compliment_list	integer	number of list compliments received by the user
compliment_note	integer	number of note compliments received by the user
compliment_plain	integer	number of plain compliments received by the user
compliment_cool	integer	number of cool compliments received by the user
compliment_funny	integer	number of funny compliments received by the user
compliment_writer	integer	number of writer compliments received by the user
compliment_photos	integer	number of photo compliments received by the user

checkin

Checkins on a business.

name	type	description
business_id	string	22 character business id, maps to business in business.json
date	string	string which is a comma-separated list of timestamps for each checkin, each with format YYYY-MM-DD HH:MM:SS

tip

Tips written by a user on a business. Tips are shorter than reviews and tend to convey quick suggestions.

name	type	description

name	type	description
date	string	when the tip was written, formatted like YYYY-MM-DD
compliment_count	integer	how many compliments it has
business_id	string	22 character business id, maps to business in business.json
user_id	string	22 character unique user id, maps to the user in user.json

Carga de datos

business

In [ ]:

```
# ID del archivo en Google Drive
file_id = '12ic-bfxm0JUjI5QWrjLjOuBm_FbvC8Fb'

# URL de descarga directa de Google Drive con gdown
file_url = f'https://drive.google.com/uc?id={file_id}'

# Descargar el archivo usando gdown
output_path = 'business.json'
gdown.download(file_url, output_path, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=12ic-bfxm0JUjI5QWrjLjOuBm_FbvC8Fb
From (redirected): https://drive.google.com/uc?id=12ic-bfxm0JUjI5QWrjLjOuBm_FbvC8Fb&confirm=
t&uuid=1d9a9168-7429-4fd8-8e02-7d8f20a42f90
To: /content/business.json
100%|██████████| 119M/119M [00:01<00:00, 102MB/s]
```

Out[ ]:

```
'business.json'
```

In [ ]:

```
%%time
output_path = 'business.json'

df_business_ini = pd.read_json(output_path, lines=True, engine='pyarrow', dtype_backend='py
arrow')

df_business_ini.head()
```

```
CPU times: user 1.97 s, sys: 749 ms, total: 2.72 s
Wall time: 234 ms
```

Out[ ]:

	business_id	name	address	city	state	postal_code	latitude	longitude	stars	review_count
0	Pns2l4eNsfO8kk83dixA6A	Abby Rappoport, LAC, CMQ	1616 Chapala St, Ste 2	Santa Barbara	CA	93101	34.427	-119.711	5.000	7
1	mpf3x-BjTdTEA3yCZrAYPw	The UPS Store	87 Grasso Plaza Shopping Center	Affton	MO	63123	38.551	-90.336	3.000	15

	business_id	name	address	city	state	postal_code	latitude	longitude	stars	review_count
2	tUFrWirKiKi_TAnsVWINQQ	Target	5255 E Broadway Blvd	Tucson	AZ	85711	32.223	-110.880	3.500	22
3	MTSW4McQd7CbVtyjqoe9mw	St Honore Pastries	935 Race St	Philadelphia	PA	19107	39.956	-75.156	4.000	80
4	mWMc6_wTdE0EUBKIGXDVfA	Perkiomen Valley Brewery	101 Walnut St	Green Lane	PA	18054	40.338	-75.472	4.500	13

In [ ]:

```
# Dimension del dataset
df_business_ini.shape
```

Out[ ]:

```
(150346, 14)
```

In [ ]:

```
df_business_ini.describe()
```

Out[ ]:

	latitude	longitude	stars	review_count	is_open
count	150346.000	150346.000	150346.000	150346.000	150346.000
mean	36.671	-89.357	3.597	44.867	0.796
std	5.873	14.919	0.974	121.120	0.403
min	27.555	-120.095	1.000	5.000	0.000
25%	32.187	-90.358	3.000	8.000	1.000
50%	38.777	-86.121	3.500	15.000	1.000
75%	39.954	-75.422	4.500	37.000	1.000
max	53.679	-73.200	5.000	7568.000	1.000

review

In [ ]:

```
# ID del archivo en Google Drive
file_id = '1NkgRjSjhK7UocCAo4LuzdBQJmTBi3Xzh'

# URL de descarga directa de Google Drive con gdown
file_url = f'https://drive.google.com/uc?id={file_id}'

# Descargar el archivo usando gdown
output_path = 'review.json'
gdown.download(file_url, output_path, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1NkgRjSjhK7UocCAo4LuzdBQJmTBi3Xzh
From (redirected): https://drive.google.com/uc?id=1NkgRjSjhK7UocCAo4LuzdBQJmTBi3Xzh&confirm=
```

```
t&uuid=dd72ae74-1c3e-4c2b-93ed-34262ac1e45d
To: /content/review.json
100%|██████████| 5.34G/5.34G [00:54<00:00, 98.1MB/s]
```

Out[ ]:

```
'review.json'
```

In [ ]:

```
%%time
output_path = 'review.json'

df_review_ini = pd.read_json(output_path, lines=True, engine='pyarrow', dtype_backend='pyarrow')

df_review_ini.head()
```

```
CPU times: user 45.1 s, sys: 26.2 s, total: 1min 11s
Wall time: 5.19 s
```

Out[ ]:

	review_id	user_id	business_id	stars	useful	funny	cool	text	
0	KU_O5udG6zpxOg-VcAEodg	mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	3.000	0	0	0	If you decide to eat here, just be aware it is...	2 0 22:0
1	BiTunyQ73aT9WBnpR9DZGw	OyoGAe7OKpv6SyGZT5g77Q	7ATYjTIgM3jUlt4UM3IypQ	5.000	1	0	1	I've taken a lot of spin classes over the year...	2 0 15:2
2	saUsX_uimxRlCVr67Z4Jig	8g_iMtfSiwikVnbP2etR0A	YjUWPpI6HXG530lwP-fb2A	3.000	0	0	0	Family diner. Had the buffet. Eclectic assortm...	2 0 20:3
3	AqPFMleE6RsU23_auESxiA	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	5.000	1	0	1	Wow! Yummy, different, delicious. Our favo...	2 0 00:0
4	Sx8TMOWLNuJBWer-0pcmoA	bcjbaE6dDog4jkNY91ncLQ	e4Vwtrqf-wpJfwesgvdgxQ	4.000	1	0	1	Cute interior and owner (?) gave us tour of up...	2 0 20:5

In [ ]:

```
# Dimension del dataset
df_review_ini.shape
```

Out[ ]:

```
(6990280, 9)
```

In [ ]:

```
df_review_ini.describe()
```

Out[ ]:

	stars	useful	funny	cool	date
count	6990280.000	6990280.000	6990280.000	6990280.000	6990280
mean	3.749	1.185	0.327	0.499	2017-01-11 11:22:33
min	1.000	-1.000	-1.000	-1.000	2005-02-16 03:23:22
25%	3.000	0.000	0.000	0.000	2015-01-25 04:53:50
50%	4.000	0.000	0.000	0.000	2017-06-03 01:26:07
75%	5.000	1.000	0.000	0.000	2019-05-23 00:02:46
max	5.000	1182.000	792.000	404.000	2022-01-19 19:48:45
std	1.479	3.254	1.689	2.172	NaN

user

In [ ]:

```
# ID del archivo en Google Drive
file_id = '1F4rJxTHQl07YF3UKjLTpjw3kErzzOj3u'

# URL de descarga directa de Google Drive con gdown
file_url = f'https://drive.google.com/uc?id={file_id}'

# Descargar el archivo usando gdown
output_path = 'user.json'
gdown.download(file_url, output_path, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1F4rJxTHQl07YF3UKjLTpjw3kErzzOj3u
From (redirected): https://drive.google.com/uc?id=1F4rJxTHQl07YF3UKjLTpjw3kErzzOj3u&confirm=
t&uuid=8e07287e-2455-4a64-a7ca-df9cb4b01308
To: /content/user.json
100%|██████████| 3.36G/3.36G [00:36<00:00, 93.2MB/s]
```

Out[ ]:

```
'user.json'
```

In [ ]:

```
%%time
output_path = 'user.json'

df_user_ini = pd.read_json(output_path, lines=True, engine='pyarrow', dtype_backend='pyarro
w')

df_user_ini.head()
```

```
CPU times: user 26.8 s, sys: 8.94 s, total: 35.7 s
Wall time: 2.19 s
```

Out[ ]:

	user_id	name	review_count	yelping_since	useful	funny	cool
0	qVc8ODYU5SZjKXVBgXdI7w	Walker	585	2007-01-25 16:47:26	7217	1259	5994

	user_id	name	review_count	yelping_since	useful	funny	cool	
1	j14WgRoU_-2ZE1aw1dXrJg	Daniel	4333	2009-01-25 04:35:42	43091	13066	27281	2009,2010,2011,2012,2013,2014,2015,201
2	2WnXYQFK0hXEoTxPtV2zvg	Steph	665	2008-07-25 10:41:00	2086	1010	1003	2009,2010,2011,
3	SZDeASXq7o05mMNLshsdIA	Gwen	224	2005-11-29 04:38:33	512	330	299	2009,
4	hA5lMy-EnncsH4JoR-hFGQ	Karen	79	2007-01-05 19:40:59	29	15	7	

In [ ]:

```
# Dimension del dataset
df_user_ini.shape
```

Out[ ]:

```
(1987897, 22)
```

In [ ]:

```
df_user_ini.describe()
```

Out[ ]:

	review_count	yelping_since	useful	funny	cool	fans	average_stars	compliment_hot	complim
count	1987897.000	1987897	1987897.000	1987897.000	1987897.000	1987897.000	1987897.000	1987897.000	198
mean	23.394	2014-10-07 07:10:01	42.296	16.971	23.793	1.466	3.630	1.807	
min	0.000	2004-10-12 08:46:11	0.000	0.000	0.000	0.000	1.000	0.000	
25%	2.000	2012-06-30 17:12:57	0.000	0.000	0.000	0.000	3.000	0.000	
50%	5.000	2014-10-12 15:09:56	3.000	0.000	0.000	0.000	3.880	0.000	
75%	17.000	2016-11-26 01:12:32	13.000	2.000	3.000	0.000	4.560	0.000	
max	17473.000	2022-01-19 17:15:47	206296.000	185823.000	199878.000	12497.000	5.000	25784.000	
std	82.567	NaN	641.481	407.803	565.351	18.131	1.183	73.602	

checkin

In [ ]:

```
# ID del archivo en Google Drive
file_id = '1syC6LXJhTgUokbPaRKLHJsR_4maPOsqm'

# URL de descarga directa de Google Drive con gdown
file_url = f'https://drive.google.com/uc?id={file_id}'

# Descargar el archivo usando gdown
output_path = 'checkin.json'
gdown.download(file_url, output_path, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1syC6LXJhTgUokbPaRKLHJsR_4maPOsqm
From (redirected): https://drive.google.com/uc?id=1syC6LXJhTgUokbPaRKLHJsR_4maPOsqm&confirm=
```

```
t&uuid=d32426a4-e0b6-446b-b427-feac9bb20771
To: /content/checkin.json
100%|██████████| 287M/287M [00:03<00:00, 79.7MB/s]
```

Out[ ]:

```
'checkin.json'
```

In [ ]:

```
%%time

output_path = 'checkin.json'

df_checkin = pd.read_json(output_path, lines=True, engine='pyarrow', dtype_backend='pyarrow')

df_checkin.head()
```

```
CPU times: user 1.26 s, sys: 175 ms, total: 1.44 s
Wall time: 112 ms
```

Out[ ]:

	business_id	date
0	---kPU91CF4Lq2-WIRu9Lw	2020-03-13 21:10:56, 2020-06-02 22:18:06, 2020...
1	--0iUa4sNDFiZFrAdIWhZQ	2010-09-13 21:43:09, 2011-05-04 23:08:15, 2011...
2	--30_8IhuyMHbSOcNWd6DQ	2013-06-14 23:29:17, 2014-08-13 23:20:22
3	--7PUidqRWpRSpXebiyxTg	2011-02-15 17:12:00, 2011-07-28 02:46:10, 2012...
4	--7jw19RH9JKXgFohspgQw	2014-04-21 20:42:11, 2014-04-28 21:04:46, 2014...

In [ ]:

```
# Dimension del dataset
df_checkin.shape
```

Out[ ]:

```
(131930, 2)
```

In [ ]:

```
df_checkin.describe()
```

Out[ ]:

	business_id	date
count	131930	131930
unique	131930	131930
top	---kPU91CF4Lq2-WIRu9Lw	2020-03-13 21:10:56, 2020-06-02 22:18:06, 2020...
freq	1	1

tip

In [ ]:

```
# df_tip = pd.read_json(data_directory+'yelp_academic_dataset_tip.json', lines=True)
# df_tip.head()
```

In [ ]:

```
# ID del archivo en Google Drive
file_id = '1uxyAM1HjqNBCwyheoo8iwl-XfS1SZrwl'

# URL de descarga directa de Google Drive con gdown
file_url = f'https://drive.google.com/uc?id={file_id}'

# Descargar el archivo usando gdown
output_path = 'tip.json'
gdown.download(file_url, output_path, quiet=False)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1uxyAM1HjqNBCwyheoo8iwl-XfS1SZrwl
From (redirected): https://drive.google.com/uc?id=1uxyAM1HjqNBCwyheoo8iwl-XfS1SZrwl&confirm=
t&uuid=b1b6381a-1448-4e72-aeac-ff9fd8ef33e5
To: /content/tip.json
100%|██████████| 181M/181M [00:01<00:00, 114MB/s]
```

Out[ ]:

```
'tip.json'
```

In [ ]:

```
%%time

output_path = 'tip.json'

df_tip = pd.read_json(output_path, lines=True, engine='pyarrow', dtype_backend='pyarrow')

df_tip.head()
```

```
CPU times: user 1.68 s, sys: 187 ms, total: 1.87 s
Wall time: 84.6 ms
```

Out[ ]:

	user_id	business_id	text	date	compliment_count
0	AGNUgVwnZUey3gcPCJ76iw	3uLgwr0qeCNMjKenHJwPGQ	Avengers time with the ladies.	2012-05-18 02:17:21	0
1	NBN4MgHP9D3cw--SnauTkA	QoezRbYQncpRqyrLH6Iqjg	They have lots of good deserts and tasty cuban...	2013-02-05 18:35:10	0
2	-copOvldyKh1qr-vzkDEvw	MYoRNLb5chwjQe3c_k37Gg	It's open even when you think it isn't	2013-08-18 00:56:08	0
3	FjMQVZjSqY8syIO-53KFKw	hV-bABTK-glh5wj31ps_Jw	Very decent fried chicken	2017-06-27 23:05:38	0
4	ld0AperBXk1h6UbqmM80zw	_uN0OudeJ3Zl_tf6nxg5ww	Appetizers.. platter special for lunch	2012-10-06 19:43:09	0

In [ ]:

```
# Dimension del dataset
df_tip.shape
```

Out[ ]:

```
(908915, 5)
```

In [ ]:

```
df_tip.describe()
```

Out[ ]:

	date	compliment_count
count	908915	908915.000
mean	2015-06-14 10:13:53	0.013
min	2009-04-16 13:11:49	0.000
25%	2013-01-26 01:18:02	0.000
50%	2015-03-15 01:10:25	0.000
75%	2017-08-02 06:19:55	0.000
max	2022-01-19 20:38:55	6.000
std	NaN	0.121

Perfilamiento

In [ ]:

```
%%time
# Para evitar una ejecución larga (>16 minutos), se incluyen los archivos generados
# en la primera ejecución por separado en la entrega final.
if False:
  # df_business.profile_report(html={'style':{'full_width':True}})
  profile_business = ProfileReport(df_business_ini)
  # guardar perfilamiento
  profile_business.to_file(output_file='profile_business.html')
```

```
CPU times: user 10 µs, sys: 6 µs, total: 16 µs
Wall time: 31.7 µs
```

In [ ]:

```
%%time
# Para evitar una ejecución larga (>24 minutos), se incluyen los archivos generados
# en la primera ejecución por separado en la entrega final.
if False:
  # df_review.profile_report(html={'style':{'full_width':True}})
  profile_review = ProfileReport(df_review_ini)
  # guardar perfilamiento
  profile_review.to_file(output_file='profile_review.html')
```

```
CPU times: user 10 µs, sys: 5 µs, total: 15 µs
Wall time: 31 µs
```

In [ ]:

```
%%time
# Para evitar una ejecución larga (>14 minutos), se incluyen los archivos generados
# en la primera ejecución por separado en la entrega final.
if False:
  # df_user.profile_report(html={'style':{'full_width':True}})
  profile_user = ProfileReport(df_user_ini)
  # guardar perfilamiento
  profile_user.to_file(output_file='profile_user.html')
```

```
CPU times: user 9 µs, sys: 5 µs, total: 14 µs
Wall time: 28.8 µs
```

In [ ]:

```
%%time
# Para evitar una ejecución larga (>2 minutos), se incluyen los archivos generados
# en la primera ejecución por separado en la entrega final.
if False:
```

```
  # df_checkin.profile_report(html={'style':{'full_width':True}})
  profile_checkin = ProfileReport(df_checkin)
  # guardar perfilamiento
  profile_checkin.to_file(output_file='profile_checkin.html')
```

```
CPU times: user 0 ns, sys: 15 µs, total: 15 µs
Wall time: 29.8 µs
```

In [ ]:

```
%%time
# Para evitar una ejecución larga (>1 minutos), se incluyen los archivos generados
# en la primera ejecución por separado en la entrega final.
if False:
  # df_tip.profile_report(html={'style':{'full_width':True}})
  profile_tip = ProfileReport(df_tip)
  # guardar perfilamiento
  profile_tip.to_file(output_file='profile_tip.html')
```

```
CPU times: user 0 ns, sys: 15 µs, total: 15 µs
Wall time: 29.8 µs
```

In [ ]:

```
# Contar interacciones por business
business_counts = df_business_ini.groupby("business_id")["stars"].count()
business_counts.describe()
```

Out[ ]:

	review_count
count	150346.000
mean	1.000
std	0.000
min	1.000
25%	1.000
50%	1.000
75%	1.000
max	1.000

dtype: double[pyarrow]

In [ ]:

```
# Contar interacciones por review
review_counts = df_review_ini.groupby("review_id")["stars"].count()
review_counts.describe()
```

Out[ ]:

	stars
count	6990280.000
mean	1.000
std	0.000
min	1.000
25%	1.000
50%	1.000

75%	1.000
max	1.000

dtype: double[pyarrow]

In [ ]:

```
x = df_review_ini.merge(
        df_business_ini,
        on='business_id'
    )

interacciones = x.groupby('business_id')[['useful', 'funny', 'cool']].sum()

# Agregamos una columna total de interacciones
interacciones['total_interacciones'] = interacciones.sum(axis=1)

# Ordenamos por total de interacciones
# interacciones = interacciones.sort_values('total_interacciones', ascending=False)

interacciones.describe()
```

Out[ ]:

	useful	funny	cool	total_interacciones
count	150346.000	150346.000	150346.000	150346.000
mean	55.078	15.183	23.183	93.444
std	138.693	54.060	79.299	255.821
min	0.000	0.000	0.000	0.000
25%	7.000	1.000	1.000	11.000
50%	19.000	4.000	5.000	29.000
75%	51.000	13.000	18.000	82.000
max	14627.000	8083.000	14132.000	32416.000

Dado la finalidad del modelo de recomendación, se usará solo los datos de `business`, `review` y `user`

Ajuste de datos

Se limita el conjunto de datos a 100,000 registros con el fin de reducir los tiempos de entrenamiento y prueba durante el desarrollo.

In [ ]:

```
# Dado el volumen de los datos, se procede a reducir la cantidad de registros a trabajar
size = 100000
```

In [ ]:

```
df_review = df_review_ini[:size].copy()
df_business = df_business_ini[:size].copy()
df_user = df_user_ini[:size].copy()
```

In [ ]:

```
# Se dejan las columnas de mayor interes para el modelo
df_business = df_business[['business_id', 'name', 'city', 'state', 'postal_code',
```

```
                              'latitude', 'longitude', 'stars', 'review_count',
                              'is_open', 'attributes', 'categories']]

df_review = df_review[['review_id', 'user_id', 'business_id', 'stars',
                           'useful', 'funny', 'cool', 'text', 'date']]

df_user = df_user[['user_id', 'name', 'review_count', 'yelping_since',
                       'useful', 'funny', 'cool', 'elite', 'fans', 'average_stars']]
```

In [ ]:

```
# Se agregan más datos de date en columnas adicionales para el df_review
df_review['year'] = df_review['date'].dt.year
df_review['month'] = df_review['date'].dt.month
df_review['day_of_week'] = df_review['date'].dt.dayofweek
df_review['hour'] = df_review['date'].dt.hour
df_review['minute'] = df_review['date'].dt.minute
```

In [ ]:

```
# Se guardan los dataset en archivos csv, con el fin de ser cargados en una tabla de base de datos

# Se omite la ejecución de esta celda para evitar sobreescribir los datos previos y/o llenar el storage
if False:
  df_business.to_csv(data_directory+'df_business.csv', index=False)
  df_checkin.to_csv(data_directory+'df_checkin.csv', index=False)
  df_review.to_csv(data_directory+'df_review.csv', index=False)
  df_tip.to_csv(data_directory+'df_tip.csv', index=False)
  df_user.to_csv(data_directory+'df_user.csv', index=False)
```

Filtrado colaborativo con Matrix Factorization (SVD)

Se utiliza SVD para generar recomendaciones basadas en las interacciones previas entre usuarios y negocios.

In [ ]:

```
# Se establece el rango en el cual se aceptaran los ratings
reader = Reader( rating_scale = ( 1, 5 ) )
```

In [ ]:

```
# Se crea el dataset a partir del dataframe
data = Dataset.load_from_df(df_review[['user_id', 'business_id', 'stars']], reader)
```

In [ ]:

```
# Se realiza el particionamiento de información en los conjuntos de entrenamiento y pruebas
from surprise.model_selection import train_test_split
train_set, test_set = train_test_split(data, test_size=0.2, random_state=seed)
testset = test_set
```

In [ ]:

```
# Número de usuarios e ítems
print("Número de user_id     :", train_set.n_users)
print("Número de business_id :", train_set.n_items)
print("Número de stars_review:", train_set.n_ratings)
```

```
Número de user_id     : 65485
Número de business_id : 9394
Número de stars_review: 80000
```

In [ ]:

```
# Cantidad del conjunto de prueba
len(test_set)
```

Out[ ]:

```
20000
```

Búsqueda de parámetros para el modelo

In [ ]:

```
# Parametros de busqueda de los mejores valores para el modelo
param_grid = {
    'n_factors': [25, 50, 100, 150, 200],
    'n_epochs': [20, 30, 50],
    'biased': [True],
    'init_mean': [0],
    'init_std_dev': [0.1],
    'lr_all': [0.002, 0.005, 0.01],
    'reg_all': [0.02, 0.05, 0.1, 0.2],
    'random_state': [seed],
    'verbose': [False]
}
```

In [ ]:

```
from surprise.model_selection import GridSearchCV
gs = GridSearchCV(SVD, param_grid, measures=['rmse','mae'], cv=5, n_jobs=-1)
gs.param_grid
```

Out[ ]:

```
{'n_factors': [25, 50, 100, 150, 200],
 'n_epochs': [20, 30, 50],
 'biased': [True],
 'init_mean': [0],
 'init_std_dev': [0.1],
 'lr_all': [0.002, 0.005, 0.01],
 'reg_all': [0.02, 0.05, 0.1, 0.2],
 'random_state': [10],
 'verbose': [False]}
```

In [ ]:

```
%%time
start_time = timer(None)
gs.fit(data)
timer(start_time)
```

```
 Time taken: 0 hour(s) 2 minute(s) and 52.76 second(s).
CPU times: user 2min 44s, sys: 2.83 s, total: 2min 46s
Wall time: 2min 52s
```

In [ ]:

```
# Visualización de datos del Cross Validation
pd.DataFrame(gs.cv_results)
```

Out[ ]:

	split0_test_rmse	split1_test_rmse	split2_test_rmse	split3_test_rmse	split4_test_rmse	mean_test_rmse	std_test_rmse	rank_t
0	**1.289**	**1.274**	**1.276**	**1.282**	**1.271**	**1.278**	**0.007**	

	split0_test_rmse	split1_test_rmse	split2_test_rmse	split3_test_rmse	split4_test_rmse	mean_test_rmse	std_test_rmse	rank_t
1	1.289	1.274	1.276	1.283	1.271	1.278	0.007	
2	1.290	1.274	1.276	1.283	1.272	1.279	0.007	
3	1.291	1.275	1.277	1.284	1.273	1.280	0.007	
4	1.268	1.254	1.257	1.264	1.250	1.259	0.007	
...	...	...	...	...	...	...	...	...
175	1.267	1.254	1.256	1.261	1.250	1.258	0.006	
176	1.267	1.254	1.257	1.262	1.251	1.258	0.006	
177	1.267	1.254	1.256	1.261	1.250	1.257	0.006	
178	1.267	1.254	1.256	1.261	1.250	1.257	0.006	
179	1.269	1.255	1.257	1.263	1.252	1.259	0.006	

180 rows × 30 columns

In [ ]:

```
best_params = gs.best_params['rmse']
best_score  = gs.best_score['rmse']

# Ver los mejores parámetros
print("Mejores parámetros encontrados: ", best_params)
print("Mejor RMSE: ", best_score)
```

```
Mejores parámetros encontrados:  {'n_factors': 25, 'n_epochs': 50, 'biased': True, 'init_mea
n': 0, 'init_std_dev': 0.1, 'lr_all': 0.005, 'reg_all': 0.1, 'random_state': 10, 'verbose':
False}
Mejor RMSE:  1.2503576437420465
```

In [ ]:

```
# Entrenar modelo con los mejores parametros
model = SVD(
    n_factors = best_params['n_factors'],
    n_epochs = best_params['n_epochs'],
    biased = best_params['biased'],
    init_mean = best_params['init_mean'],
    init_std_dev = best_params['init_std_dev'],
    lr_all = best_params['lr_all'],
    reg_all = best_params['reg_all'],
    random_state = best_params['random_state'],
    verbose = best_params['verbose']
    )
```

In [ ]:

```
%%time
start_time = timer(None)
model.fit(train_set)
timer(start_time)
```

```
 Time taken: 0 hour(s) 0 minute(s) and 1.54 second(s).
CPU times: user 1.55 s, sys: 2.36 ms, total: 1.56 s
Wall time: 1.54 s
```

In [ ]:

```
%%time
# Evaluar modelo en test
predictions = model.test(test_set)
rmse = accuracy.rmse(predictions)
mae = accuracy.mae(predictions)

print(f"Modelo SVD - RMSE: {rmse}, MAE: {mae}")
```

```
RMSE: 1.2458
MAE:  0.9972
Modelo SVD - RMSE: 1.2457718938573337, MAE: 0.9972263238576734
CPU times: user 107 ms, sys: 8.2 ms, total: 115 ms
Wall time: 111 ms
```

In [ ]:

```
# Guardar modelo
joblib.dump(model,'modelSVD.joblib')
```

Out[ ]:

```
['modelSVD.joblib']
```

Modelos sensibles al contexto (Context-Aware)

A continuación, se utiliza Random Forest como recomendador sensible al contexto, aprovechando su capacidad para considerar factores como hora, día y estación en las recomendaciones.

In [ ]:

```
# Merge entre df review y business
context_df = df_review.merge(
        df_business[['business_id', 'latitude', 'longitude', 'city', 'state']],
        on='business_id'
    )
```

In [ ]:

```
# Extracción y generación de nuevos valores de acuerdo a la hora
context_df['time_of_day'] = pd.cut(
        context_df['date'].dt.hour,
        bins=[0, 6, 12, 18, 24],
        labels=['night', 'morning', 'afternoon', 'evening'],
        include_lowest=True
    )
```

In [ ]:

```
# Extracción y generación de nuevos valores de acuerdo a la temporada basado en el mes
context_df['season'] = pd.cut(
        context_df['date'].dt.month,
        bins=[0, 3, 6, 9, 12],
        labels=['winter', 'spring', 'summer', 'fall'],
        include_lowest=True
    )
```

In [ ]:

```
# Creación de one-hot encoded de caracteristicas para variables categoricas
context_features = pd.get_dummies(
        context_df[['user_id', 'business_id', 'time_of_day', 'season', 'day_of_week', 'city
', 'state']],
        columns=['time_of_day', 'season', 'day_of_week', 'city', 'state']
    )
```

In [ ]:

```
# agregar el stars_rw como variable target
context_features['rating'] = context_df['stars']
```

In [ ]:

```
# Ver las variables basadas en contexto
context_features
```

Out[ ]:

	user_id	business_id	time_of_day_night	time_of_day_morning	time_of_day_afternoo
0	mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	False	False	Fals
1	OyoGAe7OKpv6SyGZT5g77Q	7ATYjTIgM3jUlt4UM3IypQ	False	False	Tru
2	8g_iMtfSiwikVnbP2etR0A	YjUWPpI6HXG530lwP-fb2A	False	False	Fals
3	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	True	False	Fals
4	bcjbaE6dDog4jkNY91ncLQ	e4Vwtrqf-wpJfwesgvdgxQ	False	False	Fals
...	...	...	...	...	
99995	SMH5CeiLvKx61IKwtLZ_PA	lV0k3BnsIFRkuWD_kbKd0Q	False	False	Fals
99996	2clTdtp-BjphxLjN83CpUA	G0xz3kyRhRi6oZl7KfR0pA	False	False	Fals
99997	MRrN6DH3QGCFcDv5RENYVg	C4lZdhasjZVQyDlOiXY1sA	True	False	Fals
99998	rnNQzeKJbvqVCsYsL10mkQ	dChRGpit9fM_kZK5pafNyA	False	True	Fals
99999	_BcWyKQL16ndpBdggh2kNA	hMcgO98QaOFmQVTfCUeGzw	False	True	Fals

100000 rows × 586 columns

In [ ]:

```
# Extracción de IDs unicos
user_ids = context_features['user_id'].unique()
business_ids = context_features['business_id'].unique()
```

In [ ]:

```
# Mapeo de IDs
user_id_map = {uid: i for i, uid in enumerate(user_ids)}
business_id_map = {bid: i for i, bid in enumerate(business_ids)}
```

In [ ]:

```
# Relacionar los indices de los IDs únicos con los datos en una nueva columna
context_features['user_idx'] = context_features['user_id'].map(user_id_map)
context_features['business_idx'] = context_features['business_id'].map(business_id_map)
```

In [ ]:

```
# Visualización
context_features
```

Out[ ]:

	user_id	business_id	time_of_day_night	time_of_day_morning	time_of_day_afternoo
0	mh_-eMZ6K5RLWhZyISBhwA	XQfwVwDr-v0ZS3_CbbE5Xw	False	False	Fals
1	OyoGAe7OKpv6SyGZT5g77Q	7ATYjTIgM3jUlt4UM3IypQ	False	False	Tru
2	8g_iMtfSiwikVnbP2etR0A	YjUWPpI6HXG530lwP-fb2A	False	False	Fals
3	_7bHUi9Uuf5__HHc_Q8guQ	kxX2SOes4o-D3ZQBkiMRfA	True	False	Fals
4	bcjbaE6dDog4jkNY91ncLQ	e4Vwtrqf-wpJfwesgvdgxQ	False	False	Fals
...	...	...	...	...	
99995	SMH5CeiLvKx61lKwtLZ_PA	lV0k3BnslFRkuWD_kbKd0Q	False	False	Fals
99996	2clTdtp-BjphxLjN83CpUA	G0xz3kyRhRi6oZl7KfR0pA	False	False	Fals
99997	MRrN6DH3QGCFcDv5RENYVg	C4lZdhasjZVQyDlOiXY1sA	True	False	Fals
99998	rnNQzeKJbvqVCsYsL10mkQ	dChRGpit9fM_kZK5pafNyA	False	True	Fals
99999	_BcWyKQL16ndpBdggh2kNA	hMcgO98QaOFmQVTfCUeGzw	False	True	Fals

100000 rows × 588 columns

In [ ]:

```
# Preparando features, excluyendo (excluding user_id, business_id, y rating)
feature_cols = [col for col in context_features.columns
                    if col not in ['user_id', 'business_id', 'rating', 'user_idx', 'busines
s_idx']]
```

In [ ]:

```
# Visualización de features
pd.DataFrame(feature_cols)
```

Out[ ]:

	0
0	time_of_day_night

	0
1	time_of_day_morning
2	time_of_day_afternoon
3	time_of_day_evening
4	season_winter
...	...
578	state_NC
579	state_NJ
580	state_NV
581	state_PA
582	state_TN

583 rows × 1 columns

In [ ]:

```
context_features.shape
```

Out[ ]:

```
(100000, 588)
```

In [ ]:

```
# Reducción de datos para otro modelo
df_reduced = context_features.copy()
```

In [ ]:

```
# Separar conjunto de datos
from sklearn.model_selection import train_test_split, GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(
        df_reduced[['user_idx', 'business_idx'] + feature_cols],
        df_reduced['rating'],
        test_size=0.2,
        stratify=df_reduced['rating'],
        random_state=seed
    )
```

In [ ]:

```
# Dimensiones de cada conjunto de datos
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out[ ]:

```
((80000, 585), (20000, 585), (80000,), (20000,))
```

In [ ]:

```
# Parámetros para GridSearchCV
param_grid_RF = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 15, 20],
    'min_samples_split': [20, 50, 100],
    'min_samples_leaf': [5, 20, 50],
    'max_features': ['sqrt', 'log2', 0.2, 0.3, 0.5],
    'max_samples': [0.5, 0.7, 1.0]
}
```

In [ ]:

```
from sklearn.model_selection import train_test_split, GridSearchCV
gsRF = GridSearchCV(
        RandomForestRegressor(random_state=seed, n_jobs=-1),
        param_grid_RF,
        scoring='r2',
        cv=5,
        n_jobs=-1
    )
gsRF
```

Out[ ]:

▸ GridSearchCV i ?

▸ estimator: RandomForestRegressor

▸ RandomForestRegressor ?

In [ ]:

```
%%time
start_time = timer(None)
gsRF.fit(X_train, y_train)
timer(start_time)
```

```
 Time taken: 2 hour(s) 21 minute(s) and 12.48 second(s).
CPU times: user 21min 14s, sys: 1min 37s, total: 22min 51s
Wall time: 2h 21min 12s
```

In [ ]:

```
# Visualización de Resultado de CV
pd.DataFrame(gsRF.cv_results_)
```

Out[ ]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_max_depth	param_max_features	param_max_samples
0	**3.218**	**0.417**	**2.140**	**1.074**	**10**	**sqrt**	**0.500**
1	**5.466**	**0.605**	**2.232**	**0.513**	**10**	**sqrt**	**0.500**
2	**6.733**	**0.346**	**1.619**	**0.430**	**10**	**sqrt**	**0.500**
3	**5.614**	**0.627**	**1.551**	**0.724**	**10**	**sqrt**	**0.500**
4	**9.553**	**1.260**	**2.159**	**0.781**	**10**	**sqrt**	**0.500**
...	**...**	**...**	**...**	**...**	**...**	**...**	**...**

1210	**mean_fit_time**	**std_fit_time**	**mean_score_time**	**std_score_time**	**param_max_depth**	**param_max_features**	**param_max_samples**	
1211	300.917	9.436	0.862	0.681	20	0.500	1.000	
1212	206.868	28.082	4.409	1.972	20	0.500	1.000	
1213	238.015	10.815	2.506	1.813	20	0.500	1.000	
1214	220.816	9.074	0.380	0.217	20	0.500	1.000	

1215 rows × 19 columns

In [ ]:

```
# Ver resultados
best_params = gsRF.best_params_

print("Mejores hiperparámetros:")
print(best_params)
```

```
Mejores hiperparámetros:
{'max_depth': 20, 'max_features': 0.5, 'max_samples': 1.0, 'min_samples_leaf': 5, 'min_sampl
es_split': 20, 'n_estimators': 300}
```

In [ ]:

```
gsRF.best_estimator_
```

Out[ ]:

▾ RandomForestRegressor i ?

```
RandomForestRegressor(max_depth=20, max_features=0.5, max_samples=1.0,
                      min_samples_leaf=5, min_samples_split=20,
                      n_estimators=300, n_jobs=-1, random_state=10)
```

In [ ]:

```
# Evaluación
best_model = gsRF.best_estimator_
y_pred = best_model.predict(X_test)
rmse = mean_squared_error(y_test, y_pred)
print(f"RMSE en test: {rmse:.4f}")
```

```
RMSE en test: 1.7490
```

In [ ]:

```
# Modelo de RandomForest con mejores parámetros
modelRF = RandomForestRegressor(
    **best_params,
    random_state=seed,
    n_jobs=-1)
```

```
modelRF
```

Out[ ]:

▾ RandomForestRegressor i ?

```
RandomForestRegressor(max_depth=20, max_features=0.5, max_samples=1.0,
                      min_samples_leaf=5, min_samples_split=20,
                      n_estimators=300, n_jobs=-1, random_state=10)
```

In [ ]:

```
%%time
start_time = timer(None)
modelRF.fit(X_train, y_train)
timer(start_time)
```

```
 Time taken: 0 hour(s) 0 minute(s) and 10.45 second(s).
CPU times: user 14min 16s, sys: 169 ms, total: 14min 16s
Wall time: 10.4 s
```

In [ ]:

```
%%time
# Evaluar el modelo
train_score = modelRF.score(X_train, y_train)
test_score = modelRF.score(X_test, y_test)

print(f"Modelo Context-aware - Train R²: {train_score:.4f}, Test R²: {test_score:.4f}")
```

```
Modelo Context-aware - Train R²: 0.1184, Test R²: 0.0470
CPU times: user 5.49 s, sys: 283 ms, total: 5.77 s
Wall time: 1.21 s
```

In [ ]:

```
# Analisis de importancia de variables
feature_importance = pd.DataFrame({
        'feature': ['user_idx', 'business_idx'] + feature_cols,
        'importance': modelRF.feature_importances_
    }).sort_values('importance', ascending=False)

print("Top 10 Más importantes contextual features:")
feature_importance.head(10)
```

```
Top 10 Más importantes contextual features:
```

Out[ ]:

	feature	importance
1	business_idx	0.348
0	user_idx	0.215
2	time_of_day_night	0.017
16	day_of_week_6	0.016
7	season_spring	0.016
5	time_of_day_evening	0.015
8	season_summer	0.015
6	season_winter	0.015
4	time_of_day_afternoon	0.015
9	season_fall	0.014

feature importance

In [ ]:

```
# Guardar modelo
joblib.dump(modelRF,'modelRF.joblib')
```

Out[ ]:

```
['modelRF.joblib']
```

In [ ]:

```
# Guardar datos del contexto
joblib.dump(user_id_map,'user_id_map.joblib')
joblib.dump(business_id_map,'business_id_map.joblib')
joblib.dump(feature_cols,'feature_cols.joblib')
joblib.dump(X_test,'X_test.joblib')
joblib.dump(y_test,'y_test.joblib')
joblib.dump(feature_importance,'feature_importance.joblib')
```

Out[ ]:

```
['feature_importance.joblib']
```

In [ ]:

```
# Comprimir modelo
!zip modelRF.joblib.zip modelRF.joblib
```

```
updating: modelRF.joblib (deflated 84%)
```

Hybrid Recommendation

Con los modelos previamente entrenados de SVD y RandomForest, se procede a realizar una hibridación de los modelos.

En este caso se crea una clase para leer los datos y generar una recomendación y/o explicación basado en los datos.

El modelo de CF(SVD) tiene un peso de 0.7 y el context-aware (RandomForest) tiene un peso de 0.3.

In [ ]:

```
class HybridRecommender:
    """
    Hybrid recommendation system that combines collaborative filtering and context-aware mod
els
    with optimized performance for large datasets and reduced memory footprint for serializa
tion
    """

    def __init__(self, cf_model, context_model, data_dict,
                 cf_weight=0.7, context_weight=0.3, precompute=True):
        """
        Initialize the hybrid recommender

        Args:
            cf_model: Collaborative filtering model
            context_model: Context-aware model
            data_dict (dict): Dictionary containing DataFrames
            cf_weight (float): Weight for collaborative filtering predictions
            context_weight (float): Weight for context-aware predictions
            precompute (bool): Whether to precompute and cache data for faster recommendatio
ns
```

```
        """
        self.cf_model = cf_model
        self.context_model = context_model
        self.cf_weight = cf_weight
        self.context_weight = context_weight

        # Process and efficiently store data dictionaries
        self.process_data_dict(data_dict)

        # Maps for converting between internal IDs and actual IDs
        self.raw_to_inner_user_id = cf_model.trainset._raw2inner_id_users
        self.raw_to_inner_item_id = cf_model.trainset._raw2inner_id_items
        self.inner_to_raw_user_id = cf_model.trainset._inner2raw_id_users
        self.inner_to_raw_item_id = cf_model.trainset._inner2raw_id_items

        # Cache global mean for fallback
        self.global_mean = cf_model.trainset.global_mean

        # Flag to track whether precomputed data is available
        self._precomputed = False

        # Precompute and cache user and item factors for faster access
        if precompute:
            self._precompute_factors()
            self._precompute_similarity_matrix()
            self._precompute_review_lookup()
            self._precomputed = True

    def process_data_dict(self, data_dict):
        """Process and optimize data storage"""
        self.data_dict = {}

        # Optimize business dataframe
        if 'business' in data_dict:
            # Convert to dictionary for O(1) lookups
            business_df = data_dict['business']
            self.business_lookup = {}
            for _, row in business_df.iterrows():
                business_id = row['business_id']
                self.business_lookup[business_id] = {
                    'name': row.get('name', 'Unknown'),
                    'categories': row.get('categories', ''),
                    'city': row.get('city', ''),
                    'state': row.get('state', ''),
                    'stars': row.get('stars', 0)
                }

            # Keep all business IDs in a set for fast membership testing
            self.all_business_ids = set(business_df['business_id'].unique())

            # Don't store the original dataframe - we have the lookup now
            # self.data_dict['business'] = business_df

        # Optimize review dataframe - store only what's needed for recommendations
        if 'review' in data_dict:
            # Instead of storing full dataframe, create a slim dictionary structure
            self.user_ratings = {}
            review_df = data_dict['review']

            # Reset index if it was set
            if isinstance(review_df.index, pd.MultiIndex):
                review_df = review_df.reset_index()

            for _, row in review_df.iterrows():
                user_id = row['user_id']
                business_id = row['business_id']
                stars = row['stars']
```

```
                if user_id not in self.user_ratings:
                    self.user_ratings[user_id] = {}

                self.user_ratings[user_id][business_id] = stars

            # Don't store the original review dataframe
            # self.data_dict['review'] = data_dict['review'][['user_id', 'business_id', 'st
ars']].copy()

    def _precompute_factors(self):
        """Precompute and cache user and item factors"""
        print("Precomputing user and item factors...")
        start_time = time.time()

        # We won't cache these for serialization, but will use them in the current session
        self.user_factors = {}
        for uid, iid in self.raw_to_inner_user_id.items():
            self.user_factors[uid] = self.cf_model.pu[iid]

        self.item_factors = {}
        for iid, inner_iid in self.raw_to_inner_item_id.items():
            self.item_factors[iid] = self.cf_model.qi[inner_iid]

        print(f"Factors precomputed in {time.time() - start_time:.2f} seconds")

    def _precompute_similarity_matrix(self):
        """Precompute similarity matrix for all users"""
        print("Precomputing user similarity matrix...")
        start_time = time.time()

        # Create a matrix of all user factors
        n_users = len(self.raw_to_inner_user_id)
        user_factor_matrix = np.zeros((n_users, len(self.cf_model.pu[0])))

        for uid, inner_id in self.raw_to_inner_user_id.items():
            user_factor_matrix[inner_id] = self.cf_model.pu[inner_id]

        # Compute full similarity matrix
        self.user_similarity_matrix = cosine_similarity(user_factor_matrix)

        print(f"Similarity matrix precomputed in {time.time() - start_time:.2f} seconds")

    def _precompute_review_lookup(self):
        """Precompute review lookup for faster access to ratings"""
        # We already created this in process_data_dict
        pass

    def get_cf_predictions_batch(self, user_id, business_ids):
        """Get predictions from collaborative filtering model for multiple items at once"""
        predictions = {}

        # Check if user is in training set
        if user_id not in self.raw_to_inner_user_id:
            # Return global mean for all items
            return {bid: self.global_mean for bid in business_ids}

        user_inner_id = self.raw_to_inner_user_id[user_id]

        # Use cached user factors if available, otherwise get from model
        if self._precomputed and user_id in self.user_factors:
            user_factor = self.user_factors[user_id]
        else:
            user_factor = self.cf_model.pu[user_inner_id]

        for business_id in business_ids:
            # Check if business is in training set
```

```
            if business_id in self.raw_to_inner_item_id:
                item_inner_id = self.raw_to_inner_item_id[business_id]

                # Use cached item factors if available, otherwise get from model
                if self._precomputed and business_id in self.item_factors:
                    item_factor = self.item_factors[business_id]
                else:
                    item_factor = self.cf_model.qi[item_inner_id]

                # Calculate dot product
                pred = np.dot(user_factor, item_factor) + self.global_mean
                predictions[business_id] = pred
            else:
                predictions[business_id] = self.global_mean

        return predictions

    def get_context_predictions_batch(self, user_id, business_ids, context_features):
        """Get predictions from context-aware model for multiple items at once"""
        if not context_features:
            return {}

        predictions = {}

        # Check if user exists in context model
        user_idx = self.context_model['user_id_map'].get(user_id)
        if user_idx is None:
            return {}

        # Prepare batch input for the model
        batch_features = []
        valid_business_ids = []

        for business_id in business_ids:
            business_idx = self.context_model['business_id_map'].get(business_id)
            if business_idx is not None:
                # Base features
                features = [user_idx, business_idx]

                # Add context features
                for col in self.context_model['feature_cols']:
                    if col in context_features:
                        features.append(context_features[col])
                    else:
                        features.append(0)  # Default value

                batch_features.append(features)
                valid_business_ids.append(business_id)

        if not batch_features:
            return {}

        # Create DataFrame for batch prediction
        columns = ['user_idx', 'business_idx'] + self.context_model['feature_cols']
        X_batch = pd.DataFrame(batch_features, columns=columns)

        # Perform batch prediction
        try:
            batch_predictions = self.context_model['model'].predict(X_batch)

            # Map predictions back to business IDs
            for i, business_id in enumerate(valid_business_ids):
                predictions[business_id] = batch_predictions[i]

        except Exception as e:
            print(f"Error in context prediction: {e}")
```

```
        return predictions

    def get_similar_users_for_business(self, user_id, business_id, top_n=3):
        """Get similar users who rated this business highly"""
        similar_users_text = []

        try:
            # Check if user and item exist in training set
            if user_id not in self.raw_to_inner_user_id or business_id not in self.raw_to_i
nner_item_id:
                return []

            user_inner_id = self.raw_to_inner_user_id[user_id]

            # If similarity matrix is precomputed, use it
            if self._precomputed and hasattr(self, 'user_similarity_matrix'):
                # Get similarity scores for all users using precomputed matrix
                similarities = self.user_similarity_matrix[user_inner_id]

                # Create a list of (inner_id, similarity) pairs
                user_similarities = [(i, similarities[i]) for i in range(len(similarities))
if i != user_inner_id]
            else:
                # Compute similarities on the fly
                user_factor = self.cf_model.pu[user_inner_id]
                user_similarities = []

                # Limit to a subset of users for performance
                sample_users = list(self.raw_to_inner_user_id.items())[:1000]

                for uid, inner_id in sample_users:
                    if inner_id != user_inner_id:
                        other_user_factor = self.cf_model.pu[inner_id]
                        sim = np.dot(user_factor, other_user_factor) / (np.linalg.norm(user
_factor) * np.linalg.norm(other_user_factor))
                        user_similarities.append((inner_id, sim))

            # Sort by similarity (descending)
            user_similarities.sort(key=lambda x: x[1], reverse=True)

            # Get top similar users
            top_similar_users = user_similarities[:top_n]

            # Check if they rated this item highly
            for inner_id, sim in top_similar_users:
                raw_id = self.inner_to_raw_user_id[inner_id]

                # Use the ratings lookup
                if hasattr(self, 'user_ratings') and raw_id in self.user_ratings and busine
ss_id in self.user_ratings[raw_id]:
                    user_rating = self.user_ratings[raw_id][business_id]
                    if user_rating >= 4:
                        similar_users_text.append(f"a similar user rated it {user_rating}/5
")

        except Exception as e:
            print(f"Error finding similar users: {e}")

        return similar_users_text

    def recommend(self, user_id, context_features=None, n=10, candidate_items=None, explana
tion=True):
        """
        Generate recommendations for a user with optimized performance

        Args:
            user_id (str): User ID
```

```
            context_features (dict): Contextual features
            n (int): Number of recommendations to generate
            candidate_items (list): List of candidate business IDs (if None, use all busines
ses)
            explanation (bool): Whether to include explanations

        Returns:
            list: List of recommendation dictionaries with scores and explanations
        """
        start_time = time.time()

        # Determine candidate items
        if candidate_items is None:
            candidate_items = list(self.all_business_ids)
        else:
            # Filter to make sure all candidates exist in our data
            candidate_items = [bid for bid in candidate_items if bid in self.all_business_i
ds]

        # Early exit if no candidates
        if not candidate_items:
            return []

        print(f"Generating recommendations for {len(candidate_items)} candidate items")

        # Get batch predictions from both models
        cf_scores = self.get_cf_predictions_batch(user_id, candidate_items)
        context_scores = {}

        if context_features and self.context_weight > 0:
            context_scores = self.get_context_predictions_batch(user_id, candidate_items, c
ontext_features)

        # Calculate final scores and create recommendation objects
        recommendations = []

        for business_id in candidate_items:
            cf_score = cf_scores.get(business_id, self.global_mean)
            context_score = context_scores.get(business_id)

            # Calculate final score
            if context_score is not None and self.context_weight > 0:
                final_score = (self.cf_weight * cf_score + self.context_weight * context_sc
ore)
            else:
                final_score = cf_score

            # Create recommendation object with business details
            business_details = self.business_lookup.get(business_id, {})

            rec = {
                'business_id': business_id,
                'score': final_score,
                'cf_score': cf_score,
                'context_score': context_score,
                'name': business_details.get('name', 'Unknown'),
                'categories': business_details.get('categories', ''),
                'city': business_details.get('city', ''),
                'stars': business_details.get('stars', 0)
            }

            # Don't generate explanations here to speed up the process
            recommendations.append(rec)

        # Sort recommendations by score and take top n
        recommendations.sort(key=lambda x: x['score'], reverse=True)
        top_recommendations = recommendations[:n]
```

```
        # Generate explanations for top recommendations if requested
        if explanation:
            with ThreadPoolExecutor(max_workers=min(10, n)) as executor:
                # Process explanations in parallel
                futures = []
                for rec in top_recommendations:
                    future = executor.submit(
                        self.generate_explanation,
                        user_id,
                        rec['business_id'],
                        rec['cf_score'],
                        rec['context_score']
                    )
                    futures.append((rec, future))

                # Collect results
                for rec, future in futures:
                    rec['explanation'] = future.result()

        print(f"Recommendations generated in {time.time() - start_time:.2f} seconds")
        return top_recommendations

    def generate_explanation(self, user_id, business_id, cf_score, context_score):
        """
        Generate an explanation for a recommendation

        Args:
            user_id (str): User ID
            business_id (str): Business ID
            cf_score (float): Collaborative filtering score
            context_score (float): Context-aware score

        Returns:
            dict: Explanation dictionary
        """
        explanation = {}

        # Collaborative filtering explanation
        explanation['collaborative'] = "This recommendation is based on the ratings of users
with similar preferences to yours."

        # Get similar users who rated this business highly
        similar_users_text = self.get_similar_users_for_business(user_id, business_id)
        if similar_users_text:
            explanation['similar_users'] = "Users with similar tastes to yours enjoyed this
business: " + ", ".join(similar_users_text)

        # Context-aware explanation
        if context_score is not None:
            # Get the most important context features
            top_features = self.context_model['feature_importance'].head(5)['feature'].toli
st()
            context_explanation = "This recommendation matches your current context"

            # Add specific context information if available
            context_specifics = []
            for feature in top_features:
                if feature.startswith('time_of_day_') or feature.startswith('season_') or f
eature.startswith('day_of_week_'):
                    context_specifics.append(feature.split('_', 1)[1])

            if context_specifics:
                context_explanation += " (" + ", ".join(context_specifics) + ")"

            explanation['context'] = context_explanation
```

```
        # Business attributes explanation
        business_details = self.business_lookup.get(business_id, {})
        categories = business_details.get('categories', '')

        if categories and categories != '':
            explanation['categories'] = f"This business is categorized as: {categories}"

        location = f"{business_details.get('city', '')}, {business_details.get('state', '')
}"
        explanation['location'] = f"Located in: {location}"

        return explanation

    def __getstate__(self):
        """Custom method to control what gets pickled"""
        # Start with the object's dictionary
        state = self.__dict__.copy()

        # Don't save precomputed data
        if 'user_factors' in state:
            del state['user_factors']
        if 'item_factors' in state:
            del state['item_factors']
        if 'user_similarity_matrix' in state:
            del state['user_similarity_matrix']

        # Set flag to indicate precomputed data is missing
        state['_precomputed'] = False

        # Keep model core but remove large data structures
        if 'data_dict' in state:
            # Remove potentially large dataframes
            state['data_dict'] = {}

        return state

    def __setstate__(self, state):
        """Custom method to control what happens during unpickling"""
        # Restore instance attributes
        self.__dict__.update(state)

        # Mark as not precomputed
        self._precomputed = False


def save_model(model, filename, compress=3):
    """
    Save model with compression and optimized memory usage

    Args:
        model: The model to save
        filename: Output filename
        compress: Compression level (0-9)
    """
    print(f"Saving model to {filename}...")
    joblib.dump(model, filename, compress=compress)
    print(f"Model saved. File size: {get_file_size(filename)}")


def get_file_size(filename):
    """Get human-readable file size"""
    import os
    size_bytes = os.path.getsize(filename)
    for unit in ['B', 'KB', 'MB', 'GB', 'TB']:
        if size_bytes < 1024.0:
            return f"{size_bytes:.2f} {unit}"
        size_bytes /= 1024.0
```

```
    return f"{size_bytes:.2f} PB"
```

Luego de crear la clase para la predicción, se procede a guardar la instancia del modelo para ser utilizado en una aplicación Web

In [ ]:

```
# Creación de diccionario consolidando todos los DF
data_dict = {
        'business': df_business,
        'review': df_review,
        'user': df_user
    }
```

In [ ]:

```
# Creación de un context_model con atributos para el modelo hibrido
context_model = {
        'model': modelRF,
        'user_id_map': user_id_map,
        'business_id_map': business_id_map,
        'feature_cols': feature_cols,
        'X_test': X_test,
        'y_test': y_test,
        'feature_importance': feature_importance
    }
```

In [ ]:

```
%%time
hybrid_model = HybridRecommender(model, context_model, data_dict)
```

```
Precomputing user and item factors...
Factors precomputed in 0.04 seconds
Precomputing user similarity matrix...
Similarity matrix precomputed in 29.29 seconds
Precomputing review lookup...
Review lookup precomputed in 3.60 seconds
```

In [ ]:

```
hybrid_model
```

Out[ ]:

```
<__main__.HybridRecommender at 0x7c1ec5fc7c10>
```

In [ ]:

```
# Guardar instancia del modelo, usando la funcion optimizada
save_model(hybrid_model, 'hybrid_model.joblib', compress=5)
```

Out[ ]:

```
['hybrid_model.joblib']
```

In [ ]:

```
# Comprimir modelo hibrido
!zip hybrid_model.joblib.zip hybrid_model.joblib
```

```
updating: hybrid_model.joblib (deflated 79%)
```

In [ ]:

```
# Comprimir en archivos de a 2GB
```

```
# !zip -s 2000m /content/hm /content/hybrid_model.joblib
```

Prueba

Se prueba el modelo híbrido generando 10 recomendaciones para un usuario bajo un contexto específico. Las recomendaciones combinan los resultados de SVD (0.7) y RandomForest (0.3), e incluyen una explicación del puntaje basado en similitud de usuarios y contexto.

In [ ]:

```
# Creación del recomendador optimizado
# Nota: precompute=True hará que la inicialización sea más lenta pero las recomendaciones más rápidas
recommender = HybridRecommender(
    cf_model=model,
    context_model=context_model,
    data_dict=data_dict,
    cf_weight=0.7,
    context_weight=0.3,
    precompute=True  # Importante: pre-calcular matrices para optimizar
)
```

In [ ]:

```
# Contexto actual
current_context = {
    'time_of_day_morning': 1,
    'time_of_day_afternoon': 0,
    'time_of_day_evening': 0,
    'time_of_day_night': 0,
    'day_of_week_weekday': 1,
    'day_of_week_weekend': 0,
    'season_summer': 1,
    'season_winter': 0,
    'season_fall': 0,
    'season_spring': 0
}

# Generar 10 recomendaciones
recommendations = hybrid_model.recommend(
    user_id='2WnXYQFK0hXEoTxPtV2zvg',
    context_features=current_context,
    n=10,
    explanation=True
)

# Imprimir recomendaciones
for i, rec in enumerate(recommendations):
    print(f"Recomendación #{i+1}: {rec['name']} (Score: {rec['score']:.2f})")
    print(f"  Categorías: {rec['categories']}")
    print(f"  Ubicación: {rec['city']}")
    print(f"  Explicación: {rec['explanation']['collaborative']}")
    if 'similar_users' in rec['explanation']:
        print(f"  {rec['explanation']['similar_users']}")
    if 'context' in rec['explanation']:
        print(f"  {rec['explanation']['context']}")
    print()
```

```
Generating recommendations for 100000 candidate items
Recommendations generated in 0.25 seconds
Recomendación #1: Frenchies Modern Nail Care Franklin (Score: 3.84)
  Categorías: Nail Salons, Beauty & Spas
  Ubicación: Franklin
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.
```

Recomendación #2: Bobby Sandwich Shop (Score: 3.84)
  Categorías: Sandwiches, Restaurants, Burgers, Cuban, Delis
  Ubicación: Tampa
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #3: Hendersonville Auto Brokers (Score: 3.84)
  Categorías: Car Brokers, Car Dealers, Boat Dealers, Automotive
  Ubicación: Hendersonville
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #4: Chissel Beauty Studio (Score: 3.84)
  Categorías: Hair Salons, Beauty & Spas
  Ubicación: Metairie
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #5: Eddie's Pizza (Score: 3.84)
  Categorías: Pizza, Restaurants
  Ubicación: Philadelphia
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #6: Assurant (Score: 3.84)
  Categorías: Financial Services, Insurance
  Ubicación: Wayne
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #7: Dollar Tree (Score: 3.84)
  Categorías: Shopping, Discount Store
  Ubicación: Warson Woods
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #8: Landmark Booksellers (Score: 3.84)
  Categorías: Bookstores, Local Flavor, Books, Mags, Music & Video, Shopping
  Ubicación: Franklin
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #9: Songbird (Score: 3.84)
  Categorías: Restaurants, Sandwiches, Breakfast & Brunch
  Ubicación: St. Louis
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.

Recomendación #10: Park West Grille (Score: 3.84)
  Categorías: Restaurants, Nightlife, American (New)
  Ubicación: Saint Louis
  Explicación: This recommendation is based on the ratings of users with similar preferences to yours.